Incident Response Report

02/13/2026

Prepared for: WeBeVulnerable
Prepared by: Perez Security

Executive Summary

On February 1st, 2026, 2026, the WBV system located at 10.12.0.101 was fully compromised by an external attacker originating from IP address 10.12.0.77. The intrusion began at 19:10:54 CST when the attacker exploited a command injection vulnerability in the web application file “lookup.php”. The web vulnerability allowed for system commands to be executed through unsanitized user input.

Within minutes of the initial exploitation, the attacker accessed a configuration file containing cleartext database credentials, enumerated the system user accounts via the “/etc/passwd” file, and successfully logged into the system over SSH as the user “devuser”. The attacker then leveraged sudo privileges to escalate to root at approximately 21:12 CST. Following privilege escalation, the attacker established multiple persistence mechanisms, including deployment of a web-based shell (find.php) and insertion of an SSH public key into “/root/.ssh/authorized_keys”. The attacker then modified system logs and shell history in an attempt to destroy any evidence of their activity.

This incident is classified as critical severity, as the attacker achieved root-level control of the system and demonstrated the ability to modify authentication mechanisms and system logs. While no further malicious activity beyond the time of 21:22 CST on February 1st, 2026 has been detected, the system must be considered fully compromised and untrusted. Immediate containment and remediation actions are required before the system can be safely returned to production.

Incident Analysis

The following timeline reconstructs attacker activity on the compromised WBV system using evidence from web access logs, authentication logs, wtmp records, and system artifacts.

February 1st, 2026, 19:10:54 - Initial Exploitation
The initial attack vector was discovered to be command injection through the website hosted at "http://10.12.0.101:80/". Looking at web access logs for the apache website, suspicious behavior is noted coming from the IP address 10.12.0.77, logs show the attacker looking through the website's directories, in an attempt to enumerate all directories including directories such as robots.txt. (***see figure 1***) While this does not show explicitly malicious behavior, later findings confirm that the attacker was enumerating directories in search of possible vulnerabilities to exploit.

```
10.12.0.77 - - [01/Feb/2026:07:36:13 -0600] "GET /robots.txt HTTP/1.1" 404 52
9 "-" "Mozilla/5.0 (X11; Linux x86_64; Debian GNU/Linux) AppleWebKit/537.36 (
KHTML, like Gecko) Chrome/119.0 Safari/537.36"
```

Figure 1 - Attempted enumerations of web directory such as /robots.txt

At 19:10:54, web access logs show a request originating from IP address 10.12.0.77 targeting "/lookup.php". The request contained injected shell metacharacters appended to the host parameter (***see figure 2***). Specifically, the attacker appends "%26%26+ls" which uses the ascii value of the "&" symbol to append system commands allowing the attacker to successfully inject commands and take advantage of the vulnerable lookup.php code.

```
10.12.0.77 - - [01/Feb/2026:19:10:54 -0600] "GET /lookup.php?host=example.com
+%26%26+ls HTTP/1.1" 200 849 "http://10.12.0.101/lookup.php?host=example.com"
 "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
```

Figure 2 - Initial exploitation through command injection on lookup.php

Looking at the source code of lookup.php (***see figure 3***) confirms that user input is passed directly into the system command without sanitization, allowing for successful command injection, granting the attacker remote command execution capability. This marks the initial access vector.

```
root@IR1:/var/www/wbv/public# cat lookup.php
<?php

$base = rtrim(dirname($_SERVER['SCRIPT_NAME']), '/');

$host = $_GET['host'] ?? '';

$out = null;

if ($host !== '') {

        ob_start();

            echo "<pre>";

            system("nslookup " . $host);

                echo "</pre>";

                $out = ob_get_clean();
```

Figure 3 - Vulnerability in lookup.php which allows command injection

February 1st, 2026, 19:15:24 - Sensitive File Access (Credential Disclosure)
Shortly after initial exploitation, evidence indicates that the attacker accessed the file "/var/www/wbv/app/config.php". (***see figure 4)*** This file contains database credentials stored in cleartext, including "DB_USER" and "DB_PASS" values. (***see figure 5***) Access to this file confirms that command execution was functioning successfully and that the attacker began gathering sensitive information for further exploitation.

```
10.12.0.77 - - [01/Feb/2026:19:15:24 -0600] "GET /lookup.php?host=example.com
+%26+cat+%2Fvar%2Fwww%2Fwbv%2Fapp%2Fconfig.php HTTP/1.1" 200 1692 "-" "curl/8
.17.0"
```

Figure 4 - Attacker accessing config.php file containing cleartext credentials

```
root@IR1:/var/www/wbv/app# cat config.php
<?php

define('DB_HOST', '127.0.0.1');

define('DB_USER', 'wbv');

define('DB_PASS', 'wbvpass');

define('DB_NAME', 'wbv');
?>
root@IR1:/var/www/wbv/app#
```

Figure 5 - Cleartext credentials stored in config.php

February 1st, 2026, 19:16:13 - User Enumeration
At 19:16:13, web logs show execution of a command to read "/etc/passwd". (***see figure 6***) This action allowed the attacker to enumerate the valid system user accounts.

```
10.12.0.77 - - [01/Feb/2026:19:16:13 -0600] "GET /lookup.php?host=example.com
+%26+cat+%2Fetc%2Fpasswd HTTP/1.1" 200 1713 "http://10.12.0.101/lookup.php?ho
st=example.com+%26+cat+%2Fvar%2Fwww%2Fwbv%2Fapp%2Fconfig.php" "Mozilla/5.0 (X
```

Figure 6 - Attacker using command cat /etc/passwd to enumerate users on system

Authentication logs were reviewed for SSH activities from 10.12.0.77 during the compromise window following web exploitation. While earlier failed SSH attempts from this IP are present in rotated authentication logs, no corresponding "Accepted password" or "Failed password" entries were observed in the available auth.log files for the period immediately following the command injection activities.

However, session records retrieved via the "last" command (wtmp) confirm that an interactive SSH session was successfully established as user devuser from 10.12.0.77.

February 1st, 2026, 19:16 - Successful Login as devuser (Post-Exploitation Access)
Wtmp and "last" command output show that a successful SSH session was established from the attacker IP 10.12.0.77 around the same time as the user enumeration. (***see figure 7***) Testing revealed that the credentials "wbvpass" were valid credentials for the user "devuser", which aligns with the "last" command output showing the attacker successfully logged in as devuser. This leads Perez Security to assume that the attacker likely attempted to SSH into the system with a username of each user found in /etc/passwd, along with the password credentials gained from config.php. It is important to note however that this cannot be fully confirmed due to the lack of authentication logs to support this conclusion.

```
root@IR1:~/ir# last
root     pts/0        10.12.0.25       Sat Feb  7 13:19 - still logged in
Debian-g tty1                          Sat Feb  7 13:18 - still logged in
reboot   system boot  6.12.57+deb13-am Sat Feb  7 13:17 - still running
root     pts/2        10.12.0.77       Sun Feb  1 21:18 - 21:19  (00:00)
root     pts/1                         Sun Feb  1 21:12 - 21:19  (00:07)
devuser  pts/0        10.12.0.77       Sun Feb  1 19:16 - 21:19  (02:02)
root     tty2                          Sat Jan 31 20:56 - 21:23 (1+00:27)
```

Figure 7 - Last command output showing sessions initiated by 10.12.0.77 as devuser and root

February 1st, 2026, 21:12 - Privilege Escalation via sudo
Analysis of the vim register history (***see figure 8***) contains references to commands including "sudo su" indicating that these commands were entered or referenced during file editing activity associated with privilege escalation. The unix time stamp, converted to CST, indicates that the attacker executed the command "sudo su" around the time of 21:12. This is also supported by the "last" command output entry for a root shell started by the attacker. (***see figure 7***)

```
# Registers:
"0      LINE    0
        roots   ALL=(ALL:ALL) ALL
|3,0,0,1,1,0,1770001744,"roots  ALL=(ALL:ALL) ALL"
""1     LINE    0
        exit
        id
        ls
        cat /etc/shadow
        ss -ptnl
        hostname
        cd /var/www/
        ls
        cd wbv/
        ls
        cd public/
        vi find.php
        ls
        touch /var/www/wbv/public/find.php
        sudo su
        exit
|3,1,1,1,16,0,1770002565,"exit","id","ls","cat /etc/shadow","ss -ptnl","hostn
ame","cd /var/www/","ls","cd wbv/","ls","cd public/","vi find.php","ls","touc
h /var/www/wbv/public/find.php","sudo su","exit"
```

Figure 8 - Viminfo registry showing commands ran by attacker

These artifacts confirm that the attacker leveraged sudo privileges from the compromised devuser account to elevate to root.

The use of "sudo su explains" the following root-level activity observed on the system such as the modification of root authorized keys and the remote root login, events which are detailed shortly after the attacker's next action.

February 1st, 2026, 21:15:55 - Webshell Interaction (Persistence Mechanism)
During the initial review of Apache access logs, POST requests to /find.php contained the parameter "feature=shell", (***see figure 9***) which is inconsistent with normal application functionality. Investigation into the directory containing this PHP script as well as through website and source code reconnaissance confirmed that find.php was a webshell likely created for persistence and easier access to the system. (***see figure 10***)

```
10.12.0.77 - - [01/Feb/2026:21:15:55 -0600] "POST /find.php?feature=pwd HTTP/
1.1" 200 228 "http://10.12.0.101/find.php" "Mozilla/5.0 (X11; Linux x86_64; r
v:140.0) Gecko/20100101 Firefox/140.0"
10.12.0.77 - - [01/Feb/2026:21:15:59 -0600] "POST /find.php?feature=shell HTT
P/1.1" 200 296 "http://10.12.0.101/find.php" "Mozilla/5.0 (X11; Linux x86_64;
 rv:140.0) Gecko/20100101 Firefox/140.0"
root@IR1:/var/www/wbv/app#
```

Figure 9 - Apache access log showing POST command for /find.php

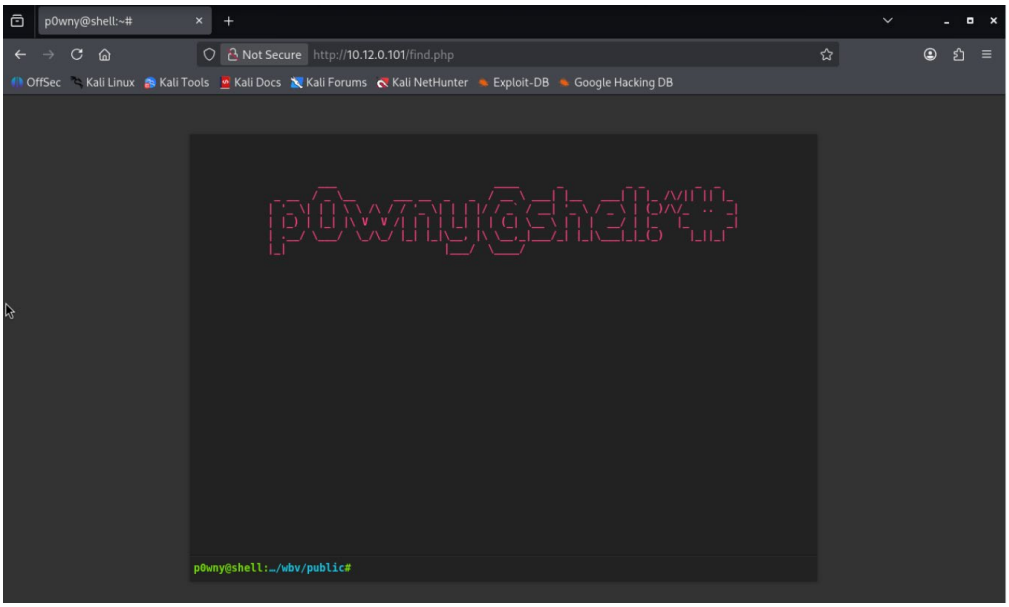

Figure 10 - Webshell established for persistence on directory /find.php

February 1st, 2026, 21:17:45 - SSH Key Persistence

Modification of "/root/.ssh/authorized_keys" was discovered to have occurred shortly after the attacker's webshell deployment. (***see figure 11***) This supports the conclusion that the attacker likely inserted their own SSH public key into the root account's authorized_keys files after elevating privileges with the sudo command. By adding their SSH public key, the attacker ensured passwordless authentication as root. This allows continued remote administrative access even if account passwords are reset.

```
root@IR1:/var/www/wbv# stat /root/.ssh/authorized_keys
  File: /root/.ssh/authorized_keys
  Size: 81              Blocks: 8          IO Block: 4096   regular file
Device: 8,1     Inode: 530425      Links: 1
Access: (0644/-rw-r--r--)  Uid: (    0/    root)   Gid: (    0/    root)
Access: 2026-02-01 21:18:44.355444542 -0600
Modify: 2026-02-01 21:17:45.847447975 -0600
Change: 2026-02-01 21:17:45.847447975 -0600
 Birth: 2026-02-01 21:17:45.847447975 -0600
```

Figure 11 - Modification times of /root/.ssh/authorized_keys

February 1st, 2026, 21:18 - Root Session Initiated

Wtmp records retrieved via the "last" command show a root session by the attacker IP (10.12.0.77) at 21:18. (***see figure 7***) The presence of this root session confirms that privilege escalation was successful and that the attacker had full administrative control of the system.

February 1st, 2026, 21:22 - Log tampering and Evidence Manipulation

Vim file marks confirm that the attacker opened and edited several critical system files around the time 21:22, referenced by the time "./bash_history" was modified. (***see figure 12***)

```
# File marks:
'0  1  1  /etc/hostname
|4,48,1,1,1770002605,"/etc/hostname"
'1  1  0  /home/devuser/.bash_history
|4,49,1,0,1770002567,"/home/devuser/.bash_history"
'2  1  0  ~/.bash_history
|4,50,1,0,1770002551,"~/.bash_history"
'3  1  79  ~/.ssh/authorized_keys
|4,51,1,79,1770002265,"~/.ssh/authorized_keys"
'4  459  0  /var/www/wbv/public/find.php
|4,52,459,0,1770002143,"/var/www/wbv/public/find.php"
'5  20402  0  /var/log/syslog
|4,53,20402,0,1770001979,"/var/log/syslog"
'6  4455  0  /var/log/auth.log
|4,54,4455,0,1770001967,"/var/log/auth.log"
'7  4455  0  /var/log/auth.log
|4,55,4455,0,1770001955,"/var/log/auth.log"
'8  50  6  /etc/sudoers.tmp
|4,56,50,6,1770001808,"/etc/sudoers.tmp"
'9  4455  0  /var/log/auth.log
|4,57,4455,0,1770001765,"/var/log/auth.log"
```

Figure 12 - File marks found on ~/.viminfo

The targeted modification of authentication logs and shell history show intentional anti-forensic behavior by the attacker, likely in an attempt to obscure the attack chain and ensure their actions could not be detected.

Compromise Assessment

The sequence of events demonstrates the complete attack sequence including initial exploitation, credential harvesting, privilege escalation, persistence establishment, and anti-forensic activity. The attacker achieved full administrative control of the WBV host and implemented multiple access mechanisms like SSH key persistence and a web-based shell, to ensure continued access.

All confirmed malicious activity originated from source IP address 10.12.0.77 and occurred within the window of February 1st, 2026 2026, between 19:10 and 21:22 CST.

Containment, Eradication, and Recovery

Containment
The first priority is stopping any further access to the compromised system while preserving evidence. The affected server (10.12.0.101) should be removed from the network immediately to prevent continued access through SSH or the deployed webshell.

Traffic from the identified attacker IP address (10.12.0.77) should be blocked at the firewall level. However, simpling blocking the IP is not sufficient since the attacker established additional persistence mechanisms.

Any active SSH session should be terminated, and remote access temporarily disabled. Before making major changes, a forensic image of the system should be captured to preserve evidence for further investigation.

The malicious webshell located at "/var/www/wbv/public/find.php" must also be removed to eliminate access to the system from a browser.

Eradication
The root cause of the compromise was the command injection vulnerability in lookup.php. The file currently passes unsanitized user input directly into the PHP "system()" function. This functionality should be removed or rewritten using proper input validation and safer execution methods

All compromised credentials must be reset. This includes the devuser account password, the root password, and the database credentials stored within config.php. The /root/.ssh/authorized_keys file must be inspected and any unauthorized public keys need to be removed. The SSH configuration should also be reviewed and hardened to disable direct root login and enforce key-based authentication with multi-factor authentication where possible. The /etc/sudoers file must also be reviewed to ensure no unauthorized modifications remain and that least-privilege access is enforced.

Because the attacker obtained root-level control and modified system logs, the integrity of the system cannot be fully trusted. For this reason, a full rebuild from a known-good backup taken prior to February 1st, 2026, 2026 is strongly recommended.

Recovery

Recovery involves restoring the system to a secure and operational state after eradication steps are taken. The safest approach is to rebuild or restore the server from a verified clean backup. After restoration, all system updates and security patches should be applied before reconnecting the system to the network. The updated web application should be reviewed and tested to ensure the command injection vulnerability has been fully resolved.

Before returning the system to production, administrators should verify that no persistence mechanisms remain. This includes confirming that no authorized SSH keys exist in any authorized_keys files, reviewing SSH configuration settings, and ensuring root SSH login is disabled.

Critical system files should be checked for integrity, and running services should be reviewed to confirm that no unauthorized processes or backdoors are active. A vulnerability scan should be performed to validate that the original weakness and any related issues have been addressed.

The system should only be placed back into service after these verification steps are completed.

Short-Term Recommendations

- Patch the command injection vulnerability in lookup.php
- Reset all system and database credentials.
- Remove unauthorized SSH keys.
- Disable direct root SSH login.
- Restrict sudo privileges to essential accounts only.

Long-term Recommendations

- Implement secure development lifecycle practices, including input validation and code review
- Deploy centralized logging to prevent local log tampering.
- Implement file integrity monitoring for critical system files.
- Enforce least privilege access controls.
- Implement multi-factor authentication for administrative access.
- Conduct regular penetration testing and vulnerability assessments.

Lessons Learned

This incident exposed several weaknesses in WBV's security posture.

The primary failure and initial compromise was made possible by insecure application development. The command injection vulnerability in "lookup.php" allowed remote command execution and served as the entry point for the attacker. Proper input validation and secure coding practices were clearly not in place when creating the application file.

Poor credential management practices also contributed to the impact. Database credentials were stored in cleartext within a web-accessible configuration file, making it easy to access once initial access was gained and to reuse for system authentication.

Access control weaknesses further increased the severity of the incident. The compromised devuser account possessed sudo privileges, allowing for immediate escalation to root. In addition, root SSH access was not sufficiently restricted.

Logging and monitoring gaps were also exposed. The attacker was able to modify authentication logs and shell history without triggering any alerts. The lack of centralized logging and integrity monitoring significantly reduced visibility during the compromise as the attacker was able to easily delete any logs or "tracks" of their actions taken on the system.

Overall, this incident highlights the importance of secure coding, proper credential management, least-privilege enforcement , and stronger monitoring controls. Implementing stronger controls in these areas would significantly reduce both the risk and impact of similar incidents in the future.